Flexibilität

Veränderte Anforderungen

- Mehr User:innen
- Zusätzliche Funktionalität
- Veränderte Betriebs-umgebung
- Komponenten austauschen (bei Kündigung z.B.)
- Veränderte Funktionalität
- "Repur-posing"
- Mehr Performance

Hindernisse

- kleine Änderung an der DB -> Deployment steht
- unbekanntes Risiko von Fehlern
- Testen teuer
- schlecht dokumentierte Contracts
- Änderung betrifft viele Komponenten
- mangelnde Wissens-verteilung
- Bottleneck andere Abteilung
- "historisch gewachsen"
- Der Monolith ist unmodular

Abhilfen

- Test-Infrastruktur
- CI/CD
- Automa-tisierung
- lose Kopplung zwischen Bausteinen
- Vertika-lisierung
- Monolith zerkleinern
- Modu-larisierungs-Techniken anwenden
- strategisches DDD
- "Hide Design Decisions"
- Entkopplung von DB-Modell und Datenmodell
- Refactoring reaktiv
- Refactoring proaktiv
- Modulsystem der Programmier-sprache
- Unterstützung anderer Abteilungen
- Wissen teilen
- Architektur-pattern
- sorgfältige Spezifikations-arbeit
- Doku-mentation

Erlbank-Aufteilung

- gemeinsame Entitäten: Konten, Kund:innen
- Gemeinsame Persistenz-schicht?
- Gemein-samkeiten bei Frontend?

1 Persistenzschicht

- 1 Admin-Team
- 1x Bereitschaftsdienst
- 1x Backups
- 1x Lizenkosten

- Abstimmung nötig
- alle gleiche DB
- alle gleiche Version
- Single Point of Failure
- Bottleneck
- Fingerpointing bei Problemen

Warten in der SW-Entwicklung

- Bottleneck andere Abteilung
- Conway's Law

Accelerate-Studie

- Warten ist schlecht
- hohe Release-Frequenz ist gut
- lose Kopplung
- schnelle Bug-Fixes gut

Schlußfolgerung: Abteilungen sollten unabhängig voneinander Releases machen können

Kopplung zwischen Bausteinen

Innerhalb des Monolithen

- gemein-samer Zustand
- Methoden-aufruf
- gemeinsame Daten
- explizite Interfaces/ Dependency Injection

Den Monolithen aufspalten

- Erwarteter Load
- Teams und Zuständig-keiten
- Wo ist der Zustand?
- nach der Fachlichkeit schneidet → nach der Technik schneiden
- Netzwerk-protokolle
- Contracts zwischen den Bausteinen

Datenintegration - über's Netzwerk

REST-API

- "Ressourcen": "Objekt mit Zustand"
- HTTP, in dem Sinn "stateless"
- jede Ressource hat eine URI
- Verben von http
-

Entity Service

- entkoppelt vom DB-Schema
- Funktionen eingeschränkt

RPC

- GraphQL
- SOAP
- gRPC

gemeinsamer Server

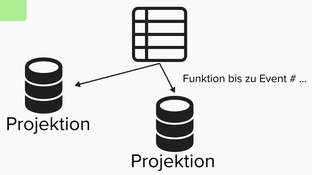
Daten duplizieren

gemein-same DB

- Transaktionen (2-Phase-Commit) -> teuer
- Konsenz-Algorithmus
- CRDTs

Themenspeicher

- Security/ Auth*n
- Was sollte gemeinsam entwickelt/ betrieben werden?
- Testen?
- Was ist mit der Daten-Konsistenz?
- Patterns für verteilte Systeme
- Misch-formen
- Observability/ Logging/ Monitoring
- Entwicklung -> Produktion/ CI
- Frontend Integration
- SCS/Events wirklich die Lösung?
- Gilden



Tradeoffs

- Versicherungs-branche
- Muß es Event-Sourcing sein?
- Laufzeit-Overhead
- Entwicklungs-Overhead

Messaging

- asynchron/ synchron
- konkreter Adressat vs. Broadcast
- Event
  - Beschreibung eines Ereignisses
  - "Fakt"
  - in der Vergangenheit
  - gut: fachlich
- Command
  - Beschreibung eines Wunsches
  - in der Zukunft
  - könnte nicht eintreten
- Erlang-Messages
- Akka
- Message-Bus
- Message-Feed
- Streaming
- Message-Broker (MQTT)
- Transportiert Zustand?
- Event-Sourcing
- "alles steht in persistierten Events"
- Idempotenz
- exactly-once delivery
- TCP/IP
- Events: Broadcast vs. 1:1
- auf Kopplung achten

Transfers: braucht "Event" über Kontoerzeugung (perspektivisch Kontoschließung)

Statements auch Kontoerzeugung, Infos über Transfers

Gemeinsame Infrastruktur

- Event-Auslieferung und Abo ?
- DB: eher nein
- einfache Domänen-objekte: nein
- Erlbank: Startguthaben :(
- technische Libraries: auf Bottlenecks achten
- Domänen-spezifische Library: eher nein
- Domänen-Kernmodell: vielleicht
- DSL für Mikro-services

-> Produkt-Team besser als Dienstleister

- Plattform-Team

- VMs
- K8s-Cluster
- Datenbank-Instanzen
- CI-Infrastruktur
- Bug-Tracker
- ...

Event-Zustellung

- Es wird Partitionen geben
- Retry (gestaffelt)
- "Kafka"
- horizontale Skalierung
- verfrühte Events zurückhalten
- Push vs. Pull
- verspätete Events?
- sich entschuldigen
- Snapshot anfertigen & cache
- nachträglich ausdünnen
- immer alles von vorn?

Events

Accounts | Transfers | Statements

- Account creation
- Transfer
- Person creation

Entwicklung -> Produktion

- CI-Pipelines
- Container
- Kubernetes
- Stage-Umge-bungen
- Service-Mesh
- interne Releases
- automa-tisierte Tests
- manuelle Tests
- statische Analyse
- immutal Build
- deklarativer Build
- SBOM
- Pinning
- festes Base Image
- Entwicklungs-rechner → Git-Repo → CI → Container